Using plate solves to calibrate the BNO055 IMU — assessment + implementation plan

Date: 2026-07-21 **Question:** Plate-solve results could calibrate the IMU while it is in use. The BNO055 has no flash to persist calibration across power cycles, so any calibration would have to be saved externally. Investigate and assess.

TL;DR

- The plate-solve→IMU calibration loop the question imagines **already exists and runs live**, in two forms: absolute attitude **anchoring** (`imu_ref`) and the camera↔IMU **extrinsic** fit (`imu_frame_R` , Kabsch). The real gap is **persistence** — not a missing calibration.
- The highest-value thing to persist is the **plate-solve-derived extrinsic** (`imu_frame_R`), which is physically fixed but re-learned from scratch every power cycle. This is exactly the "no flash → save it" idea, pointed at the right target.
- Persisting the **BNO055's own accel/gyro offset profile** (the classic no-flash workaround) is worth doing but **lower** value: in IMUPLUS mode only gyro/accel matter, the chip re-derives them anyway, and re-anchoring already bounds drift. Plate solves act as the *validator/gate*, not the source, for this profile.
- **Do not** use plate solves to re-enable the magnetometer / NDOF — the interference is uncalibratable and plate solves already are the absolute heading reference.

Status (updated 2026-07-21)

- **Units A + B are implemented and shipped in v0.11.60** (PR #162): the camera↔IMU extrinsic persists and self-heals, and the BNO055 profile restore/save is available behind `imu_persist_bno055` (default off).
- **Unit C** (below) — the promotion of Opportunity 3 into a full **accel-bias / gyro-scale calibration derived from solve-vs-IMU disagreement**: the calibration the mounted chip cannot otherwise produce. **Mode 1 (software correction) is now implemented** on the working branch behind `imu_solve_cal_enabled` (default off), pending on-sky validation: `diofinder/imu_solve_cal.py` (pure estimator + transforms, unit-tested in `tests/test_imu_solve_cal.py`), solver-side observe/estimate/persist, and the comms `_imu_predict` correction. **Mode 1 shipped in v0.11.61.**
- **Mode 2 (chip-offset write) is implemented on the working branch but NOT released** (`imu_solve_cal_write_chip` , default off, **requires bench verification**): `imu_proc` converts the estimated tilt bias to BNO055 accel-offset LSB (`imu_solve_cal.accel_offset_delta_lsb`) and adds it to the chip's registers in a CONFIG-mode excursion (the Unit B channel), then a handshake resets the solver estimator so it re-measures the post-write residual. **Two datasheet assumptions must be confirmed on a real BNO055 before trusting it** — `ACCEL_OFFSET_LSB_PER_MS2` (100 LSB per m/s²) and `ACCEL_OFFSET_SIGN` (whether the chip adds or subtracts the stored offset; flip to +1 if a bench test shows the tilt growing instead of shrinking).

Current state (as of v0.11.59)

1. The BNO055 runs in IMUPLUS mode — magnetometer disabled

`imu_proc.py` initialises the chip in `_OPR_IMUPLUS` (accel + gyro fusion, mag off) deliberately: the magnetometer is unreliable near the telescope's metal body and motor drives. Consequences:

- **Pitch/roll** are gravity-referenced → absolute, no drift.
- **Heading/yaw** has no absolute reference → drifts ~1–5°/hr from gyro bias.
- Only **accel + gyro** calibration is meaningful. Gyro calibrates by holding still a few seconds; accel from a few static orientations. **Mag calibration is moot.**

2. No BNO055 calibration registers are touched; nothing IMU-derived persists

`imu_proc.py` never reads the calib-status register (0x35) or the offset/radius blob (0x55–0x6A); the chip auto-calibrates internally each power-up. The only IMU config keys are `imu_rate_gate_dps` and `imu_exact_predict` (`config.py:265,272`). `imu_frame_R` , `imu_calib_C` , and `imu_ref` live only in `shared_cfg` and are **rebuilt from scratch every session**.

3. Plate solves already calibrate the IMU — two live mechanisms

- **Absolute attitude anchoring** (`imu_ref` , `solver_proc.py::_imu_update_reference` , ~line 809): every successful solve re-anchors the IMU dead-reckoning to the solved attitude. Drift never accumulates beyond one inter-solve interval. This is loosely-coupled plate-solve↔IMU fusion, already shipped.
- **Camera↔IMU extrinsic** (`imu_frame_R` , Kabsch fit in `imu_frame.py` ; legacy 2-D `imu_calib_C`): fit live from `(r_imu, r_sky)` rotation-vector pairs harvested between consecutive solves, quality-gated (≥4 magnitude-consistent pairs, axis diversity ≥0.25, $R^2 \geq 0.9$). The mounting rotation `M` (IMU body → camera) satisfies `r_sky = R · r_imu` ; the exact-quaternion LX200 prediction (`comms_proc._imu_predict`) uses it. This is literally "plate solves calibrating the IMU mounting as it is used."

So the plate-solve→IMU calibration already runs. What it lacks is **memory**: the mounting fit and any sensor bias are discarded on every power cycle, and the extrinsic only becomes observable *after* the scope has slewed in ≥2 non-collinear directions with a good solve at each end — before that the hint path uses the wider-cone body-frame fallback.

Opportunities, ranked

Opportunity 1 — persist the plate-solve-derived extrinsic (`imu_frame_R`). Highest value.

The IMU→camera mounting `M` is physically fixed (changes only on remount) yet is re-learned every session and is unavailable until the first multi-direction slew converges. Persisting it makes the exact-quaternion prediction available **from the first solve after boot**.

- **What to save:** the 9-float `imu_frame_R` , its quality dict (`n` , `r2` , `axis_diversity`), and a timestamp.
- **Where:** `/var/lib/diofinder/imu_extrinsic.json` , using the same atomic write discipline as `seeing_overrides.json` / the hot-pixel mask.
- **Self-heal (load-bearing):** treat the loaded value as a **seed**, keep the live Kabsch fit running, and override the stored value when the live fit is good and diverges from it beyond a threshold (a remount must recover automatically). Same philosophy as the FOV drift-recommit.
- **Plate-solve role:** direct — this artifact *is* a plate-solve product.

Opportunity 2 — persist the BNO055 accel/gyro calibration profile. Medium value.

The classic no-flash workaround, and the literal thing the question raises. Once the chip reports gyro+accel calib status = 3, read the 22-byte blob in CONFIG mode, save it, and write it back at next boot before switching to IMUPLUS. diofinder's init already does a CONFIG→IMUPLUS excursion, so restore slots in cleanly *before* the mode switch.

- **Benefit:** skips the cold-start gyro-bias warm-up so dead-reckoning is trustworthy immediately after boot.
- **Honest caveats:** in IMUPLUS only gyro/accel offsets are used (mag offsets in the blob are unused); the chip re-zeroes gyro whenever the scope sits still (constantly, between slews); and re-anchoring already bounds drift. So this is a warm-up nicety, not a correctness fix.
- **Plate-solve role:** *validator/gate*, not source. Plate solves don't feed the chip's internal accel/gyro cal (that comes from gravity/motion), but the agreement between IMU deltas and solved deltas is the right signal for "this profile is good, save it."

Opportunity 3 — plate-solve gyro-bias feed-forward → promoted to Unit C.

The original narrow idea — estimate a slowly-varying gyro-bias vector from the IMU-vs-solve delta mismatch and de-drift the between-solve prediction — is low value on its own (re-anchoring resets the reference every solve, so over a ~10 s interval even 5°/hr is only a few arcmin). But it under-sold the real prize: the same solve-vs-IMU disagreement also contains the **static accelerometer tilt error**, which *is* a genuine, invertible calibration the mounted chip can't produce on its own. That broader idea is specified below as **Unit C**; the gyro-bias piece rides along as its dynamic (non-persisted) term.

Anti-recommendation — do not re-enable the magnetometer / NDOF.

Mag is off because of hard-iron interference (scope metal, motors): a spatially-varying, non-constant field no calibration can fix. Plate solves already are the absolute heading reference, so NDOF would inject drift-prone noise for zero benefit.

Implementation plan

Two independent, additive units. Ship **Unit A first** (higher value, no hardware-register risk); **Unit B** is optional polish.

Unit A — persist + self-heal the camera↔IMU extrinsic

New module `diofinder/imu_persist.py` (pure, hardware-free, unit-testable):

```
# imu_persist.py
_PATH = "/var/lib/diofinder/imu_extrinsic.json"

def save_extrinsic(R9, quality, path=_PATH): ...   # atomic temp-file + os.replace
def load_extrinsic(path=_PATH): ...                # -> (R9, quality, mtime) | None
def clear_extrinsic(path=_PATH): ...
def extrinsic_diverged(R_live, R_saved, tol_deg=2.0) -> bool:
    # relative rotation angle between the two 3x3 matrices; > tol => remount
```

Reuse the atomic-write helper already used for `seeing_overrides.json` (do not re-implement temp-file/`os.replace`).

`solver_proc.py` — seed on boot, persist on good fit, self-heal on divergence:

1. **Boot seed.** Where the solver initialises IMU state, call `load_extrinsic()`; if present, publish `shared_cfg["imu_frame_R"] = R9` and mark it provisional (`imu_frame_quality = {"source": "persisted", ...}`) so the exact-prediction path in comms is live from the first solve.
2. **Persist on good fit.** In `_imu_update_reference`, when the live Kabsch fit produces a fresh `R9` whose quality clears a *stricter* save gate (e.g. `n >= 8`, `r2 >= 0.97`, `axis_diversity >= 0.4` — tighter than the *use* gate so only a well-observed mounting is written), and it differs from the last-saved value, call `save_extrinsic(R9, quality)`. Throttle writes (e.g. at most once per N solves or when quality improves) to avoid disk churn.
3. **Self-heal.** If a persisted seed is in force and a good live fit (`extrinsic_diverged(R_live, R_seed)` true) disagrees beyond `tol_deg`, adopt the live fit and overwrite the file — this recovers a remount without user action, mirroring the FOV drift-recommit.

Factory reset. Add `imu_extrinsic.json` to the artifacts `diofinder-factory-reset --clear-*` can delete (alongside overrides / hot-pixel mask).

Tests (`tests/test_imu_persist.py`, hardware-free): round-trip save/load; `extrinsic_diverged` true/false at known angles; atomic-write leaves no partial file on simulated failure; the stricter save gate rejects a marginal fit.

No comms change required — `_imu_predict` already consumes `imu_frame_R` transparently; it simply starts finding it populated at boot.

Unit B — persist + restore the BNO055 accel/gyro profile

`imu_proc.py` — register I/O around the existing CONFIG→IMUPLUS init:

1. **Add register constants:** calib-status `0x35`, calib blob `0x55`–`0x6A` (22 bytes: accel/mag/gyro offsets + accel/mag radii).
2. **Restore before the IMUPLUS switch.** In `_probe_and_init`, after entering `_OPR_CONFIG` and before writing `_OPR_IMUPLUS`, if `/var/lib/diofinder/bno055_calib.json` exists, block-write its 22 bytes to `0x55`. (Writing calib registers is only legal in CONFIG mode — the current sequence is already in CONFIG at that point.)
3. **Save when good, gated by plate solves.** In the poll loop, periodically read the calib-status byte; when **gyro and accel bits both read 3** *and* `shared_cfg` shows recent solves whose IMU deltas agree with solved deltas (reuse the `imu_frame_quality` / recent-solve signal — the plate-solve gate), read the 22-byte blob (requires a brief CONFIG excursion: switch to CONFIG, read, switch back to IMUPLUS — ~30 ms, do it at most once per session or when status first reaches 3) and write it via the same atomic JSON helper.
4. Expose `imu_calib_status` (sys/gyro/accel) in `shared_cfg` for the web UI / `version` /status surfaces, and a maint `imu_calib_clear` to delete the file.

Factory reset. Add `bno055_calib.json` to the clear list.

Risk notes for Unit B: the CONFIG excursion briefly stops fusion output — schedule it only when `imu_available` and the scope is idle (no active slew), and never during an align hold. A stale/temperature-drifted profile is still a valid *seed*; the chip re-converges, so a bad restore self-corrects. Keep Unit B behind a config flag (`imu_persist_bno055`, default off until field-validated).

Unit C — online accel-bias / gyro-scale calibration from solve-vs-IMU residuals

Goal. Derive the *static* IMU calibrations the mounted BNO055 cannot self-produce — **accelerometer bias (absolute tilt)** and **gyro scale-factor** — from the accumulated disagreement between plate solves and IMU output, and apply them so the *standalone* IMU (between solves, mid-slew, and when solving fails) is trustworthy. The accel tilt calibration, not the gyro drift, is the prize: the chip normally gets it from a 6-orientation tumble that is impossible once the sensor is bolted to the scope, and the plate solve **directly observes absolute tilt**, so it is the most observable thing in the residual.

Principle — split the residual into calibratable vs not. For each solve, form the IMU-vs-truth attitude error and split it about the local vertical:

- the **tilt** component (rotation about a horizontal axis) is driven by the *static* accel bias → **calibratable** (a fixed offset/coefficient exists);
- the **azimuth/yaw** component (about vertical) is the *dynamic* gyro heading drift → **not** a fixed calibration (a random walk); project it out — the live re-anchoring owns it — and never persist it as a coefficient.

A constant accel bias `b_a` tilts the sensed gravity vector, and its attitude-error signature varies predictably with the sensor's orientation relative to gravity. Many `(orientation, tilt-residual)` pairs across different **altitudes** least-squares-solve for `b_a` (3 params; an optional 3×3 scale/misalignment is a later richer model). The gyro scale-factor is the ratio `|IMU rotation delta| / |solve rotation delta|` over slews (reusing the magnitude data `imu_frame` already harvests).

Observation model (forming the residual).

1. `q_sky` — solved camera attitude (celestial J2000).
2. Site (`cfg` latitude/longitude) + time (UTC → LST) convert `q_sky` to the **local-gravity** (topocentric alt/az/parallactic) frame → `q_truth_grav` : the camera's true orientation relative to local vertical.
3. `q_imu` — BNO055 fused body quaternion (already gravity-referenced); apply the **Unit A extrinsic** to map body→camera → `q_imu_grav` .
4. Residual `r = q_truth_grav ⊖ q_imu_grav` , split into tilt (horizontal-axis) and yaw (vertical-axis) using the local vertical.

Observability gating (load-bearing; mirrors Unit A's axis-diversity gate).

- **Tilt diversity:** the gravity direction in the body frame must span a range (solves at different altitudes) or `b_a` is under-observed — second-singular- value threshold, refuse below it and stay at the seed.
- **Requires Unit A:** a good extrinsic must be present, else the tilt residual is confounded by mounting error. Hard dependency A → C. (A is observed from *relative* slew pairs, C's accel bias from *absolute* tilt residuals — so the two are separable by construction, not circular.)
- **Requires valid site/time:** refuse if latitude/longitude are `(0, 0)` (the factory-reset default) or the clock is implausible — a wrong site silently biases "truth."
- **Static/dynamic separation:** the accel fit uses the **tilt-only** residual (insensitive to yaw drift by construction); optionally a small filter carries an accel-bias (static) + gyro-yaw-bias (random-walk) state.

Where the correction applies — two staged modes.

- **Mode 1 (default, safe): software post-correction.** Apply the accel-bias tilt correction (and gyro scale) to the IMU quaternion in diofinder's consumers (`_imu_predict` / hint / dead-reckoning). Never touches the chip; fully reversible; no risk to the shipped fusion.
- **Mode 2 (opt-in, later): write to the chip.** Convert the derived accel offset to the BNO055's raw-LSB offset units and write it via the **Unit B** CONFIG-mode channel, so the chip's own fusion improves — literally feeding the chip the calibration it couldn't tumble for. Higher risk (unit conversion; the chip's auto-cal may nudge it). Behind `imu_solve_cal_write_chip` (default off) and requires `imu_persist_bno055` .

Persistence + self-heal. The estimate (accel bias, gyro scale, observability / quality, timestamp) persists via `imu_persist.py` → `/var/lib/diofinder/imu_solve_cal.json` ; seeded at boot (the Mode-1 correction is live immediately), refined online, and self-heals like Unit A (a good new estimate that diverges overwrites). Cleared by factory reset `--clear-imu-calib` .

Where it lives. Reuse the solver's existing per-solve harvest in `_imu_update_reference` (it already holds `q_now` , the solved RA/Dec/roll, `imu_ref` , and the extrinsic). Add the celestial→gravity conversion (site+time), the residual split, accumulation, and a **throttled** estimator run (every N solves); publish `shared_cfg["imu_solve_cal"]` + quality; the comms hint/predict path applies the Mode-1 correction. The estimator core (residual split, accel-bias least-squares, gyro-scale ratio, diversity gate) is **pure numpy, hardware-free** — `tests/test_imu_solve_cal.py` : inject a known accel bias into synthetic solves and recover it; verify injected yaw drift does **not** leak into the accel estimate; under-diversity is refused; gyro-scale recovered.

Config keys.

Key	Default	Notes
`imu_solve_cal_enabled`	`false`	Master switch — estimation + Mode-1 software correction.
`imu_solve_cal_write_chip`	`false`	Mode-2 chip-offset write (requires `imu_persist_bno055`).
`imu_solve_cal_min_tilt_spread_deg`	(tuned)	Observability gate on altitude spread.

Risks / honest caveats.

- **Value framing:** at a solve the benefit is **zero** (re-anchored to truth). The gain is a better *standalone* IMU — smaller between-solve / mid-slew tilt error and a better fallback when solving fails (cloud, lost-in-space). It is **not** a pointing-accuracy fix at the moment of a solve.
- **Site/time dependence:** garbage site ⇒ garbage "truth"; the validity gate is mandatory (and factory reset zeros lat/long).
- **Observability:** needs altitude spread; a near-meridian-only session simply stays at the seed — non-destructive.
- **Temperature (an advantage):** accel bias drifts with temperature, and the *online* estimator tracks it continuously — better than a one-shot tumble or a stale persisted profile.
- **Mode-2 chip write stays off** by default; Mode 1 delivers most of the value with none of the raw-unit risk.

Dependencies: requires **Unit A** (extrinsic frame); complements **Unit B** (Mode 2 writes what B persists); independent of the magnetometer decision.

Sequencing / rollout

1. Unit A — **shipped** (v0.11.60): pure software, self-healing.
2. Unit B — **shipped** (v0.11.60) behind `imu_persist_bno055=false`, pending a couple of nights confirming the CONFIG excursion is non-disruptive and the restore shortens warm-up.
3. Unit C — **next**, behind `imu_solve_cal_enabled=false`. Land Mode 1 (software correction) first with the observability + site/time gates; add Mode 2 (chip-offset write) only after Mode 1 is field-validated.

What this buys

- **Cold start after boot:** exact-quaternion pointing prediction available from the first solve (Unit A) instead of after the first multi-direction slew; trustworthy dead-reckoning immediately (Unit B) instead of after gyro warm-up.
- **Standalone IMU that keeps improving (Unit C):** the mounted accel converges toward truth from the sky without ever being tumbled, so pointing degrades gracefully between solves and when solving fails.
- **No new failure mode for the shipped path:** every unit is a seed the live machinery already overrides; the live Kabsch fit and re-anchoring remain the source of truth, and Unit C's default is a reversible software correction.

What the three units accomplish — separately and jointly

Unit	Derives / stores	Data owner	Plate solves' role	Default	Depends on
A	camera↔IMU **mounting extrinsic** (`imu_frame_R`)	diofinder (software)	**the source** (Kabsch on slew pairs)	on (self-heal)	—
B	BNO055 **accel/gyro offset blob** (22 B)	the chip	**gate / validator** (when to save)	off (`imu_persist_bno055`)	—
C	static **accel-bias + gyro-scale** correction	diofinder (Mode 1) / the chip (Mode 2)	**the source** (solve-vs-IMU residual)	off (`imu_solve_cal_enabled`)	A (+ B for Mode 2)

Separately:

- **Unit A — geometry ("where is the IMU aimed relative to the camera").** Persists the fixed mounting so the exact pointing prediction is live from the first solve after boot, and self-heals a remount.
- **Unit B — memory ("remember the chip's own zeroing").** Lets the BNO055's hard-won accel/gyro calibration survive power cycles despite having no flash — including a deliberate *pre-mount* accel tumble captured once and restored forever.
- **Unit C — learning ("derive the zeroing the chip couldn't get, from the sky").** Turns the standing disagreement between solves and the IMU into the static accel/gyro calibration the mounted chip cannot self-produce, and corrects the IMU output so it is trustworthy on its own.

Jointly (the closed loop):

1. **A provides the frame** that lets C attribute a tilt residual to the *sensor* rather than the *mounting* — without A, C's residual is confounded and meaningless.
2. **C turns ongoing disagreement into an improving calibration**, and in Mode 2 writes that accel offset back into the chip's registers…
3. **…which B then persists** across power cycles.

Net: a **self-calibrating, self-persisting IMU**. A is the geometry, B is the memory, C is the learning. Layer-wise, **A and C are diofinder-side calibrations derived from solves (software), B is the chip-side calibration, and C's Mode 2 is the bridge software→chip that B then remembers.** Together the mounted BNO055 converges toward truth without ever being tumbled and keeps what it learns across reboots — which matters precisely where the IMU has to carry pointing on its own: between solves, mid-slew, and when plate solving is unavailable.

Testing Unit C on-sky (Mode 1)

Unit C ships **off**. Validate it over a few clear nights before trusting it.

Preconditions (all three must hold, or nothing is estimated)

1. **Unit A extrinsic has converged.** Unit C refuses without a good `imu_frame_R`. Confirm via `diofinder-ctl raw '{"cmd":"status"}'` → `imu.frame_quality` shows an `r2` (not a `rejected` reason). If absent, slew the scope through **two non-collinear directions** with a solve at each end first (that is what makes the mounting observable).

2. **Correct site AND clock.** The zenith comes from latitude/longitude + UTC, so a wrong site or a wrong Pi clock poisons the "truth." Check `grep -E 'latitude|longitude' /etc/diofinder/diofinder.conf` (must **not** be 0/0) and `date -u` on the Pi (must be correct — a field unit with no network may have a bad clock; set it before testing).
3. **IMU present** (`imu.available` true).

Enable

Easiest (v0.11.62+): **Camera page → Experimental A/B card → "IMU accel calibration (Unit C)"** checkbox — live, no restart, and the readout beside it shows the estimate as it converges. Equivalent CLI/conf:

```
diofinder-ctl raw '{"cmd":"solver_params_set","args":{"imu_solve_cal_enabled":true,"persist":true}}'
# or, conf + restart:
sudo sed -i 's/^imu_solve_cal_enabled:.*/imu_solve_cal_enabled: true/' \
  /etc/diofinder/diofinder.conf   # add the line if missing
sudo systemctl restart diofinder
```

Procedure (the key is ALTITUDE diversity)

The accel bias is only observable if the gravity direction moves in the sensor frame — i.e. you must solve at **a spread of altitudes**, not just near the meridian. Over a session:

1. Point low (~20–30° alt), get solves; point mid; point high (near zenith); repeat across a few directions. Aim for ≥ 8–10 solves spanning altitude.
2. Watch it converge — the **Camera-page readout** (`tilt · r² · obs · min_eig · gyro×`) updates live every ~3 s. Or from the shell: `bash journalctl -u diofinder -f | grep -i "Unit C solve-cal" # Unit C solve-cal: tilt=1.83 deg r2=0.985 min_eig=2.10 n=14 gyro_scale=1.002` or one-shot: `diofinder-ctl raw '{"cmd":"status"}'` → `imu.solve_cal` .

What "good" looks like

- `min_eig` clears the gate (published at all ⇒ it did) and rises with altitude spread — this is the observability metric.
- `tilt` is **small and stable** across the session and night-to-night (a plausible accel bias is well under a few degrees). `r2` high (> ~0.9).
- With the correction on, the SkySafari crosshair between solves / mid-slew is as good or better. **If it looks worse, disable and it reverts** (set `imu_solve_cal_enabled: false` , restart — the correction is software-only).

Red flags (and what they mean)

Symptom	Likely cause
Nothing ever logged / `solve_cal` null	Unit A extrinsic missing, or site is 0/0
`min_eig` stays low, no publish	not enough altitude spread — point across more sky
`tilt` huge (> ~10°) or unstable session-to-session	wrong clock/site, or a frame-convention mismatch (the part needing field confirmation)
Crosshair worse with it on	disable; capture a bundle (below)

Debug bundle to send if you need help

If the estimate looks wrong or the crosshair degrades, capture **while the test session is live** (Unit C enabled, having solved across altitudes):

1. **A debug bundle** from the Web UI → *Debug* (or `curl http://diofinder.local/debug/collect?frames=12 -o bundle.zip`). It now carries the Unit A/B/C state in `imu.json` (`frame_quality` , `solve_cal` , `calib_status`) plus the frame burst, `effective_params.json` , and the conf.
2. **The session journal** covering the run: `journalctl -u diofinder --since "today" > diofinder.log` — the `Unit C solve-cal:` lines show the convergence history.
3. **The persisted artifacts**: `/var/lib/diofinder/imu_solve_cal.json` and `/var/lib/diofinder/imu_extrinsic.json` (the extrinsic Unit C depends on).
4. **Confirm site+clock**: paste `date -u` and the `latitude` / `longitude` conf lines — the single most common cause of a wrong estimate.

With those four, the tilt estimate, the extrinsic it was built on, the exact frames/attitudes, and the site/time can all be replayed to isolate a convention vs. an observability vs. a site/clock problem.

Bench-verifying Mode 2 before ever enabling it

Mode 2 writes to the chip, so validate the two datasheet constants on a bench BNO055 **before** `imu_solve_cal_write_chip: true`, and only after Mode 1's tilt estimate is trusted:

1. With Mode 1 converged to a stable non-trivial tilt, note it.
2. Enable Mode 2 for **one** write (watch `journalctl` for `Unit C Mode 2: wrote accel offset delta … LSB`), then let Mode 1 re-measure.
3. **The residual tilt must SHRINK.** If it grows by roughly the same amount, the offset sign is inverted — flip `imu_solve_cal.ACCEL_OFFSET_SIGN` to `+1`. If it changes by the wrong magnitude, `ACCEL_OFFSET_LSB_PER_MS2` is off.
4. Confirm the written offsets survive a power cycle (they ride in the Unit B `bno055_calib.json`, so `imu_persist_bno055` must also be on).

Until that bench loop passes, Mode 2 stays off — Mode 1's software correction delivers the pointing benefit with none of the chip-write risk.

Bottom line

The premise is right — plate solves calibrate the IMU, and no on-chip flash means saving externally — but the leverage is inverted from the obvious reading. The plate-solve→IMU loop already runs live; **Unit A** (shipped) persists its most valuable product, the camera↔IMU extrinsic, instead of relearning it every boot; **Unit B** (shipped, opt-in) remembers the chip's own accel/gyro zeroing across power cycles. The remaining prize is **Unit C**: the accel-tilt (and gyro-scale) calibration the mounted chip can't otherwise produce is *derivable* from the solve-vs-IMU disagreement — the static part is a real, invertible calibration; only the heading drift is not, and that stays a running estimate (or re-anchoring). A magnetometer re-enable remains off the table.